1/23/2025	9 AM - 10 AM	Advance SQL Query

Evaluate previous class.

1/23/2025	10 AM - 11 AM	**Select top nth record**

Find the nth record:

Select Top 1 Salary from (

Select distinct top n salary from Employee order by salary desc) as result order by asc

Select * from (

Select id, name, otherColumn Dense_Rank() over (order by salary) as Ranking from Employee) as temp

Where Ranking=5

To select the **n-th largest number** from a table in SQL Server, you can use the ROW_NUMBER() or RANK() window functions, combined with ORDER BY to sort the data in descending order and assign a unique rank to each row. Then, you can filter for the specific rank corresponding to the n-th largest number.

Using ROW_NUMBER() to Select the n-th Largest Number

Here's how you can do it with ROW_NUMBER():

Example:

Assume you have a table numbers with the following data:

number

10

20

30

number

40

50

To get the 3rd largest number, you would use the ROW_NUMBER() function:

sql

```
WITH RankedNumbers AS (
  SELECT number, ROW_NUMBER() OVER (ORDER BY number DESC) AS RowNum
  FROM numbers
)
SELECT number
FROM RankedNumbers
WHERE RowNum = 3;
```

Explanation:

1. **ROW_NUMBER() OVER (ORDER BY number DESC)**: This function assigns a unique row number to each row in the table, ordered by the number column in descending order. The largest number gets row number 1, the second-largest gets row number 2, and so on.
2. **WITH RankedNumbers**: This creates a temporary result set (common table expression or CTE) where each row is ranked.
3. **WHERE RowNum = 3**: This filters the result set to return the row with the 3rd largest number.

Using RANK() or DENSE_RANK()

Alternatively, you can use RANK() or DENSE_RANK() if you want to account for ties in the data. These functions behave similarly to ROW_NUMBER(), but they handle duplicate values differently.

- **RANK()** will leave gaps in the ranks if there are ties.
- **DENSE_RANK()** will not leave any gaps if there are ties.

Example using RANK():

sql

```
WITH RankedNumbers AS (
   SELECT number, RANK() OVER (ORDER BY number DESC) AS RankNum
   FROM numbers
)
SELECT number
FROM RankedNumbers
WHERE RankNum = 3;
```

This works the same as ROW_NUMBER(), but in the case of ties, the same rank is assigned to equal values, and the next rank skips accordingly.

Example with Ties (RANK vs. DENSE_RANK):

Assume this table with numbers, where 30 and 40 have duplicate values:

number
10
20
30
40
40
50

Using RANK() to find the 3rd largest number:

sql

```
WITH RankedNumbers AS (
   SELECT number, RANK() OVER (ORDER BY number DESC) AS RankNum
   FROM numbers
)
SELECT number
```

```
FROM RankedNumbers
WHERE RankNum = 3;
```

- **RANK()** will return both 30 and 40 as the 3rd largest because they share the same rank (and 40 appears twice).
- The next rank after 30 and 40 will be 5 (skipping rank 4).

Using DENSE_RANK() to find the 3rd largest number:

sql

```
WITH RankedNumbers AS (
  SELECT number, DENSE_RANK() OVER (ORDER BY number DESC) AS RankNum
  FROM numbers
)
SELECT number
FROM RankedNumbers
WHERE RankNum = 3;
```

- **DENSE_RANK()** will return 30 as the 3rd largest number, and 40 will still be ranked 2nd, without skipping ranks.

Conclusion:

To select the **n-th largest number** in SQL Server:

- Use ROW_NUMBER() if you just need the n-th record ordered by size.
- Use RANK() or DENSE_RANK() if you need to account for ties in the data.

For the 3rd largest number:

- **ROW_NUMBER()**: No ties are handled.
- **RANK()**: Ties will cause a gap in rankings.
- **DENSE_RANK()**: Ties are handled without gaps in rankings.

1/23/2025	11 AM - 12 PM

Create Cluster Index

Create Cluster Index in SQL Server

In SQL Server, a **Clustered Index** is an index that determines the physical order of the rows in a table. Essentially, the data in the table is stored in the order of the clustered index. Each table can have only one clustered index because the data rows themselves can only be sorted in one order.

How to Create a Clustered Index in SQL Server:

You can create a clustered index using the CREATE CLUSTERED INDEX statement. Typically, you would create a clustered index on a column that is frequently used for sorting or as a primary key, such as an ID column.

Here is the general syntax to create a clustered index:

sql

CREATE CLUSTERED INDEX index_name

ON table_name (column_name [ASC|DESC]);

- index_name: The name of the clustered index.
- table_name: The name of the table on which you want to create the clustered index.
- column_name: The column(s) that the index will be created on. You can specify one or multiple columns.
- ASC or DESC: Specifies whether the index will be sorted in ascending or descending order. The default is ASC.

Example 1: Create a Clustered Index on a Single Column

Suppose you have a table Employees with the following structure:

EmployeeID	FirstName	LastName	DateOfBirth
1	John	Doe	1990-01-01

EmployeeID	FirstName	LastName	DateOfBirth
2	Alice	Smith	1985-05-15
3	Bob	Johnson	1988-09-30

To create a clustered index on the EmployeeID column (which is often a good choice for a clustered index, especially if it's unique and used for sorting), you would write:

sql

```
CREATE CLUSTERED INDEX IX_EmployeeID
ON Employees (EmployeeID);
```

Example 2: Create a Clustered Index on Multiple Columns

You can also create a clustered index on multiple columns. This is called a composite clustered index. For example, if you want to create a clustered index on both LastName and FirstName:

sql

```
CREATE CLUSTERED INDEX IX_EmployeeName
ON Employees (LastName, FirstName);
```

In this case, the rows will be physically ordered by LastName first, and then FirstName second.

Example 3: Creating a Clustered Index When Defining the Table

You can also define a clustered index when you first create the table by using the PRIMARY KEY or UNIQUE constraint, as they automatically create a clustered index on the specified column(s).

Example: Create a Clustered Index on the Primary Key:

sql

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,  -- Automatically creates a clustered index
    FirstName NVARCHAR(50),
```

```
    LastName NVARCHAR(50),
    DateOfBirth DATE
);
```

Here, the PRIMARY KEY constraint on EmployeeID automatically creates a clustered index on this column.

Example 4: Drop an Existing Clustered Index

If you need to drop an existing clustered index, use the DROP INDEX command:

sql

```
--------
DROP INDEX IX_EmployeeID ON Employees;
```

Important Considerations:

- **Only One Clustered Index**: A table can have only one clustered index because the rows can only be physically ordered in one way.
- **Primary Key**: By default, a primary key constraint creates a clustered index, unless one already exists. In that case, the primary key will create a non-clustered index.
- **Performance**: Creating a clustered index on a column that is frequently used for sorting or searching can significantly improve query performance. However, the clustered index can affect the performance of INSERT, UPDATE, and DELETE operations, as the rows need to be physically rearranged when those operations occur.
- **NULL Values**: Clustered indexes will place NULL values at the end of the index if you're using ASC ordering, and at the beginning if you're using DESC ordering.

Conclusion:

A **clustered index** is crucial for improving the performance of queries that sort or filter by the indexed column(s). By default, creating a **primary key** or **unique constraint** will automatically create a clustered index. Otherwise, you can manually create a clustered index using the CREATE CLUSTERED INDEX command, either on a single column or multiple columns depending on your needs.

1/23/2025	12PM - 1 PM	Practice Session

1/23/2025	2 PM - 3 PM	Delete Duplicate Record

Delete Duplicate Record in SQL Server

To **delete duplicate records** in SQL Server, you can use a combination of the CTE (Common Table Expression), the ROW_NUMBER() function, and a DELETE statement. The basic idea is to assign a unique row number to each duplicate record and then delete the rows that are considered duplicates.

Here's how you can do it step by step:

Steps to Delete Duplicate Records

1. **Use the ROW_NUMBER() function**: This will help assign a unique number to each row within a partition of data (grouped by the columns that determine duplicates). By sorting the data, we can assign numbers to the duplicates and identify which ones to keep.
2. **Identify and delete duplicates**: Using a CTE (Common Table Expression), you can mark the rows to be deleted by identifying all but one of the duplicate rows.

Example 1: Delete Duplicate Rows Based on All Columns

Let's say you have a table Employees with columns EmployeeID, FirstName, LastName, and DateOfBirth. If there are duplicate records (where all columns are the same), you can use the following method:

SQL Query to Delete Duplicate Records:

sql

```
WITH CTE AS (
  SELECT
    EmployeeID, FirstName, LastName, DateOfBirth,
    ROW_NUMBER() OVER (PARTITION BY FirstName, LastName, DateOfBirth ORDER BY EmployeeID) AS RowNum
  FROM Employees
)
```

```
DELETE FROM CTE
WHERE RowNum > 1;
```

Explanation:

1. **ROW_NUMBER()**: This function assigns a unique sequential number to each row, ordered by EmployeeID. The PARTITION BY clause groups the rows by FirstName, LastName, and DateOfBirth, which are the columns used to identify duplicates. The ORDER BY EmployeeID ensures that the row with the smallest EmployeeID is considered the "original" record and is not deleted.
2. **WITH CTE AS (...)**: This CTE creates a temporary result set where each duplicate row is numbered. The RowNum column will have a value of 1 for the first occurrence of each duplicate set, and higher numbers for subsequent duplicates.
3. **DELETE FROM CTE WHERE RowNum > 1**: This part deletes all rows where the RowNum is greater than 1, which means only the first occurrence of each duplicate set is kept.

Example 2: Delete Duplicate Records Based on Some Columns (Partial Duplicate)

If you want to delete duplicates based only on a subset of columns (e.g., based only on FirstName and LastName), and you want to keep the record with the smallest EmployeeID, you can adjust the query:

sql

```
WITH CTE AS (
  SELECT
    EmployeeID, FirstName, LastName, DateOfBirth,
    ROW_NUMBER() OVER (PARTITION BY FirstName, LastName ORDER BY EmployeeID) AS RowNum
  FROM Employees
)
DELETE FROM CTE
WHERE RowNum > 1;
```

In this case:

- Duplicates are identified based on the FirstName and LastName columns.

- The row with the smallest EmployeeID (i.e., RowNum = 1) is kept, and all other duplicates are deleted.

Example 3: Delete Duplicates Based on a Unique Column (e.g., EmployeeID)

If you have duplicates and want to delete them based on a specific column but want to retain a unique identifier (e.g., keeping the row with the smallest EmployeeID), you can modify the query like so:

sql

```
WITH CTE AS (
  SELECT
    EmployeeID, FirstName, LastName, DateOfBirth,
    ROW_NUMBER() OVER (PARTITION BY FirstName, LastName, DateOfBirth ORDER BY EmployeeID) AS RowNum
  FROM Employees
)
DELETE FROM Employees
WHERE EmployeeID IN (SELECT EmployeeID FROM CTE WHERE RowNum > 1);
```

Explanation:

- The CTE creates a temporary result set, and the query filters to delete all duplicate rows with RowNum > 1.
- The main table Employees is referenced, and the rows to be deleted are matched using the EmployeeID values from the CTE.

1/23/2025	3 PM - 4 PM	Rank()

The RANK() function in SQL is a **window function** that assigns a unique rank to each row within a result set, based on a specific order. The rank is assigned sequentially, starting at 1. If there are ties (i.e., multiple rows having the same value in the ordered column), the RANK() function assigns the same rank to those rows, but it will leave gaps in the ranking sequence for subsequent rows.

For example, if two rows are tied for rank 1, the next row will be assigned rank 3 (skipping rank 2).

Syntax of RANK():

sql

RANK() OVER (PARTITION BY partition_column ORDER BY order_column [ASC|DESC])

- **PARTITION BY**: (Optional) Divides the result set into partitions (groups) to apply the rank function separately within each group. If you omit this, the rank is applied to the entire result set.
- **ORDER BY**: Specifies the order in which to rank the rows. The RANK() function assigns ranks based on this order.
- **ASC or DESC**: Specifies the order (ascending or descending). The default is ASC (ascending).

Example Usage:

Example 1: Basic Example of RANK()

Consider a table Sales with the following data:

EmployeeID	SalesAmount
1	1000
2	1500
3	1500
4	1200
5	900

To assign ranks based on the SalesAmount in descending order, you can use the RANK() function as follows:

sql

```
SELECT
  EmployeeID,
  SalesAmount,
```

```
    RANK() OVER (ORDER BY SalesAmount DESC) AS SalesRank
FROM Sales;
```

Result:

EmployeeID	SalesAmount	SalesRank
2	1500	1
3	1500	1
4	1200	3
1	1000	4
5	900	5

Explanation:

- Employees 2 and 3 are tied for the top rank because they have the same SalesAmount of 1500, so both receive rank 1.
- The next rank is 3 (skipping rank 2) because of the tie.
- The rank numbering skips 2, demonstrating that RANK() leaves gaps when there are ties.

Example 2: Using RANK() with PARTITION BY

If you want to rank employees within each department, you can use the PARTITION BY clause. Assume there is an additional DepartmentID column in the Sales table:

EmployeeID	SalesAmount	DepartmentID
1	1000	1
2	1500	1
3	1500	2
4	1200	2
5	900	1

To rank employees by SalesAmount within each DepartmentID, use the PARTITION BY clause:

sql

```
SELECT
  EmployeeID,
  SalesAmount,
  DepartmentID,
  RANK() OVER (PARTITION BY DepartmentID ORDER BY SalesAmount DESC) AS SalesRank
FROM Sales;
```

Result:

EmployeeID	SalesAmount	DepartmentID	SalesRank
2	1500	1	1
1	1000	1	2
5	900	1	3
3	1500	2	1
4	1200	2	2

Explanation:

- The RANK() function assigns ranks separately for each department due to the PARTITION BY DepartmentID.
- For DepartmentID = 1, employees 2, 1, and 5 are ranked 1, 2, and 3, respectively.
- For DepartmentID = 2, employees 3 and 4 are ranked 1 and 2.

Example 3: Handling Ties with RANK()

The RANK() function is useful when you want to preserve ties and leave gaps in the rank numbering. However, if you don't want gaps in your rankings, you can use the DENSE_RANK() function instead.

For example, with the same table as above (Sales), when employees 2 and 3 tie for rank 1:

sql

```
--------
SELECT
  EmployeeID,
```

```
    SalesAmount,
    RANK() OVER (ORDER BY SalesAmount DESC) AS SalesRank,
    DENSE_RANK() OVER (ORDER BY SalesAmount DESC) AS DenseSalesRank
FROM Sales;
```

Result:

EmployeeID	SalesAmount	SalesRank	DenseSalesRank
2	1500	1	1
3	1500	1	1
4	1200	3	2
1	1000	4	3
5	900	5	4

Explanation:

- The RANK() function leaves a gap after the tied rows, so the next rank is 3 (skipping rank 2).
- The DENSE_RANK() function, however, does not leave any gaps. It continues numbering the ranks sequentially (1, 1, 2, 3, 4), even after ties.

Conclusion:

The RANK() function in SQL is useful when you need to assign rankings to rows, and it's especially helpful when there are ties in the ordered column(s). If you want to maintain a continuous sequence of ranks without gaps, you should use DENSE_RANK() instead of RANK(). You can also use PARTITION BY to create rankings within groups of data.

1/23/2025	4 PM - 5 PM	**Dense_Renk()**

The DENSE_RANK() function in SQL is similar to the RANK() function, but with a key difference in how it handles ties. While RANK() leaves gaps in the rank sequence when there are ties (duplicates in the data), DENSE_RANK() does **not** leave any gaps, meaning the ranking continues consecutively even when there are ties.

Syntax of DENSE_RANK():

sql

DENSE_RANK() OVER (PARTITION BY partition_column ORDER BY order_column [ASC|DESC])

- **PARTITION BY**: (Optional) Divides the result set into partitions (groups) to apply the DENSE_RANK() function separately within each group. If omitted, the function ranks the entire result set.
- **ORDER BY**: Specifies the order in which the rows are ranked. The DENSE_RANK() function assigns ranks based on this order.
- **ASC or DESC**: Specifies the sorting order (ascending or descending). By default, it is ASC (ascending).

Key Difference Between RANK() and DENSE_RANK():

- **RANK()**: If two rows have the same value, they will both receive the same rank, but the next rank will skip one. For example, if two rows tie at rank 1, the next row will be ranked 3 (skipping rank 2).
- **DENSE_RANK()**: If two rows have the same value, they will both receive the same rank, but the next rank will be assigned to the very next number, with no gaps in the rank sequence.

Example Usage of DENSE_RANK():

Consider the following Sales table:

EmployeeID	SalesAmount
1	1000
2	1500
3	1500
4	1200
5	900

Example 1: Ranking SalesAmount in Descending Order Using DENSE_RANK()

To assign a rank to each SalesAmount in descending order, you would use:

sql

```
SELECT
  EmployeeID,
  SalesAmount,
  DENSE_RANK() OVER (ORDER BY SalesAmount DESC) AS SalesRank
FROM Sales;
```

Result:

EmployeeID	SalesAmount	SalesRank
2	1500	1
3	1500	1
4	1200	2
1	1000	3
5	900	4

Explanation:

- Employees 2 and 3 have the same SalesAmount of 1500, so they are both ranked 1.
- The next rank after 1 is 2 (not 3, as with RANK()), meaning employee 4 is ranked 2, even though employee 4's sales amount is lower.
- The ranks continue without any gaps.

Example 2: Using DENSE_RANK() with PARTITION BY

If you want to rank sales within each department, you can use the PARTITION BY clause. Assume there is an additional DepartmentID column in the Sales table:

EmployeeID	SalesAmount	DepartmentID
1	1000	1
2	1500	1
3	1500	2
4	1200	2

EmployeeID	SalesAmount	DepartmentID
5	900	1

To rank employees within each department based on SalesAmount, you would write:

sql

```
SELECT
  EmployeeID,
  SalesAmount,
  DepartmentID,
  DENSE_RANK() OVER (PARTITION BY DepartmentID ORDER BY SalesAmount DESC) AS SalesRank
FROM Sales;
```

Result:

EmployeeID	SalesAmount	DepartmentID	SalesRank
2	1500	1	1
1	1000	1	2
5	900	1	3
3	1500	2	1
4	1200	2	2

Explanation:

- The DENSE_RANK() function assigns ranks separately within each department (PARTITION BY DepartmentID).
- For DepartmentID = 1, the sales amounts are ranked as 1, 2, and 3.
- For DepartmentID = 2, the sales amounts are ranked as 1 and 2.

When to Use DENSE_RANK():

- **No Gaps in Ranking**: Use DENSE_RANK() when you want to assign consecutive ranks without leaving gaps, even when there are ties.

- **Grouped Ranking**: It is useful when partitioning data (e.g., for each department, category, etc.) and you want ranks to continue in a consecutive sequence without skipping.

Difference Between DENSE_RANK() and RANK():

Function	Handling Ties	Rank Sequence
RANK()	Assigns the same rank to tied rows, but skips subsequent ranks.	Gaps in rank sequence
DENSE_RANK()	Assigns the same rank to tied rows, but does not skip subsequent ranks.	No gaps in rank sequence

Conclusion:

The DENSE_RANK() function is a useful tool for ranking rows in SQL while ensuring that tied rows receive the same rank and the rank numbering continues sequentially without any gaps. It is often used when you need to maintain a continuous rank sequence even in the presence of duplicates.

1/23/2025	5 PM - 6 PM	Pagination

Pagination in SQL Server is the process of retrieving a subset (or "page") of rows from a larger result set, typically used for displaying data in a user-friendly manner (e.g., showing 10 rows at a time on a webpage).

SQL Server provides multiple methods to implement pagination, with the most common approaches being:

1. **Using OFFSET-FETCH (introduced in SQL Server 2012)**
2. **Using ROW_NUMBER() with a Common Table Expression (CTE)**

1. Pagination using OFFSET-FETCH (SQL Server 2012 and later)

The OFFSET-FETCH clause is the most straightforward way to implement pagination in modern versions of SQL Server (2012 and later). It allows you to specify how many rows to skip and how many rows to return.

Syntax:

sql

SELECT columns

FROM table

ORDER BY column

OFFSET (page_number - 1) * page_size ROWS

FETCH NEXT page_size ROWS ONLY;

- **OFFSET**: Specifies the number of rows to skip before starting to return rows. For example, for page 2, you would skip (page_number - 1) * page_size rows.
- **FETCH NEXT**: Specifies the number of rows to return, which is typically the page size.

Example:

Assume you have a table Employees and you want to retrieve **page 2** of results, with each page containing **5 records**:

sql

SELECT EmployeeID, FirstName, LastName, SalesAmount

FROM Employees

ORDER BY SalesAmount DESC

OFFSET 5 ROWS -- Skip the first 5 rows

FETCH NEXT 5 ROWS ONLY; -- Fetch the next 5 rows (i.e., page 2)

Explanation:

- **OFFSET 5 ROWS** skips the first 5 rows (i.e., page 1).
- **FETCH NEXT 5 ROWS ONLY** returns the next 5 rows (i.e., page 2).

Handling Dynamic Pagination (Variables):

You can use variables for dynamic pagination, for example:

sql

DECLARE @PageNumber INT = 2;

DECLARE @PageSize INT = 5;

```
SELECT EmployeeID, FirstName, LastName, SalesAmount
FROM Employees
ORDER BY SalesAmount DESC
OFFSET (@PageNumber - 1) * @PageSize ROWS
FETCH NEXT @PageSize ROWS ONLY;
```

This allows you to change the page number and size dynamically.

1/23/2025	6 PM - 7 PM	Practice Session